



Universidad Nacional Autónoma de México
Facultad de Ingeniería



**Computación Gráfica e Interacción
Humano-Computadora**

Proyecto Final
Entorno Virtual 3D

Tema

“Five Nights at Freddy's”

Maestro:

Edén Urzua Espinoza

Integrantes:

Calderón Gómez Guadalupe Nikole - 320303115 - Gpo.Teoría. 3
González De Luna Berenice - 320075999 - Gpo.Teoría. 3

Fecha de Entrega:

11 de mayo de 2026

Índice

1. Objetivos
2. Alcance y Limitantes
3. Requerimientos y Ambiente
4. Arquitectura y Estructura del Proyecto (Visual Studio sin CMake)
5. Metodología de Desarrollo
6. Diagrama de Flujo del Software
7. Plan y Diagrama de Gantt
8. Documentación del Código
9. Diseño del Escenario 3D (Geometría, Texturizado, Iluminación, Animación)
10. Pruebas y Validación
11. Gestión de Riesgos y Mantenimiento
12. Conclusiones y Trabajo Futuro

Manual Técnico

1. Objetivos.

1.1 Objetivo general.

Desarrollar un entorno virtual 3D interactivo inspirado en Five Nights at Freddy's, utilizando OpenGL en C++, donde el usuario pueda recorrer o visualizar una escena de terror ambientada en una pizzería oscura, con iluminación dinámica, modelos 3D, texturas, cámara interactiva y animaciones básicas.

1.2 Objetivos específicos.

1. Modelar la geometría base del escenario.

Crear una escena 3D que represente una pizzería estilo FNAF, incluyendo elementos como oficina de seguridad, pasillos, puertas, mesas, sillas, escenario, cámaras de vigilancia, posters, luces y objetos decorativos.



2. Aplicar texturas y materiales diferenciados.

Usar texturas para paredes, piso, madera, metal, carteles, mesas, puertas, buscando una atmósfera de abandono y suspenso.

3. Implementar iluminación.

Agregar luces para reforzar el ambiente de terror.

4. Cargar modelos externos con Assimp.

Integrar modelos 3D externos, como animatrónicos, muebles, cámaras, puertas u objetos decorativos, usando archivos .obj.

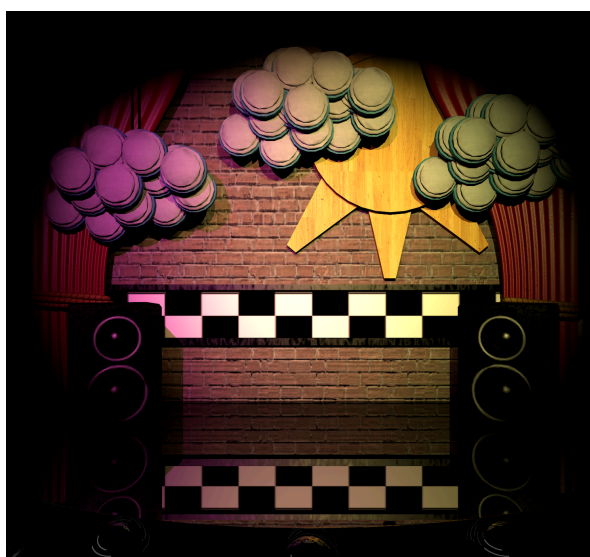


5. Permitir navegación e interacción básica.

Implementar navegación en primera persona mediante teclado y mouse, permitiendo la exploración interactiva del entorno.

6. Agregar animaciones.

Incluir movimientos simples como puertas abriéndose, pelota rebotando, nubes y sol subiendo y bajando, y otros más complejos como animatrónicos moviéndose.





7. Construir una experiencia visual coherente con la temática.

Lograr que la escena transmita una atmósfera oscura, misteriosa y de suspenso, sin copiar directamente el juego, sino creando una interpretación académica inspirada en su estilo.

2. Alcances y limitantes.

El proyecto consiste en el desarrollo de un entorno virtual 3D interactivo inspirado en la estética de Five Nights at Freddy's, implementado en C++ mediante OpenGL moderno. La aplicación permite visualizar y recorrer una escena ambientada en una pizzería oscura, incorporando modelos 3D, texturas, iluminación, animaciones básicas e interacción mediante cámara en primera persona.

El escenario incluye distintos elementos visuales y decorativos, como pasillos, puertas, mesas, sillas, objetos temáticos y personajes animados. Además, se implementan sistemas básicos de iluminación y animación para generar una experiencia visual más dinámica e inmersiva dentro del entorno virtual.

2.1 Alcances.

- Desarrollo de un entorno virtual 3D renderizado en tiempo real utilizando OpenGL 3.3 Core Profile.
- Integración de modelos externos en formato .obj mediante la librería Assimp.
- Implementación de iluminación direccional, puntual y spotlight mediante shaders GLSL.
- Uso de texturas y materiales para mejorar la apariencia visual del escenario y los modelos.
- Implementación de cámara en primera persona con navegación mediante teclado y mouse.
- Incorporación de animaciones básicas utilizando transformaciones geométricas y variables dependientes del tiempo.
- Organización modular del proyecto mediante clases para shaders, cámara, modelos y renderizado.
- Compatibilidad con sistemas Windows utilizando Visual Studio 2022 y arquitectura x64.

2.2 Limitantes técnicas.

1. **Sin inteligencia artificial avanzada.**
Los animatrónicos no tendrán comportamiento inteligente real. Sus movimientos serán predefinidos, por posiciones, rotaciones y trayectorias.
2. **Sin físicas realistas.**
No se implementará un motor de físicas. No habrá gravedad realista, colisiones complejas, rebotes físicos ni simulaciones avanzadas.
3. **Colisiones simples o inexistentes.**
El usuario podría atravesar paredes, mesas u objetos si no se implementa un sistema básico de colisiones.
4. **Iluminación básica/intermedia.**
Se utilizarán luces pero no se contemplan sombras dinámicas complejas, HDR, SSAO ni iluminación global.
5. **Animaciones sencillas.**
Las animaciones serán matemáticas o por transformaciones básicas: apertura de puertas, movimiento de animatrónicos o cambios de posición.
6. **Modelos y texturas optimizados para rendimiento.**
Se usarán modelos de complejidad moderada y texturas reducidas para mantener buen desempeño en equipos comunes.
7. **Sin jumpscare complejos.**
No se puede incluir cinemáticas elaboradas, audio ni lógica de juego completa.
8. **Sin copia exacta del juego original.**
El proyecto será una escena inspirada en la temática de FNAF, pero no una réplica exacta del mapa, personajes o mecánicas oficiales. El entorno está diseñado principalmente con fines académicos y demostrativos, por lo que no busca replicar de forma exacta el juego original.
9. **Compatibilidad limitada.**
El proyecto estará pensado para ejecutarse en PC con Windows, Visual Studio 2022, OpenGL 3.3 Core y librerías configuradas correctamente.
10. **Equipos utilizados.**
El rendimiento visual puede variar dependiendo de la capacidad gráfica y de procesamiento del equipo utilizado.

3. Requerimientos y Ambiente.

Para la correcta compilación y ejecución del proyecto es necesario contar con un entorno de desarrollo compatible con OpenGL moderno, así como con las librerías necesarias para el manejo de gráficos, modelos 3D, texturas e interacción en tiempo real. El proyecto fue desarrollado y probado en Windows utilizando Visual Studio 2022 y arquitectura x64.

3.1 Software de desarrollo.

- **Visual Studio 2022.**

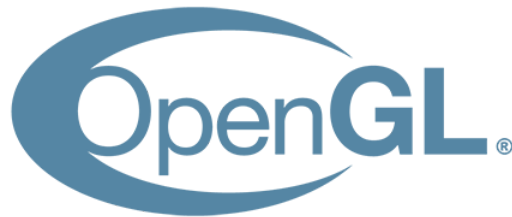
- Configuración en x64.
- Toolset MSVC v143 o superior.
- Proyecto configurado para OpenGL 3.3 Core.



3.2 API gráfica

El proyecto utiliza OpenGL moderno basado en shaders programables y renderizado en tiempo real.

- **OpenGL 3.3 Core Profile.**
 - Uso de shaders GLSL.
 - Pipeline gráfico moderno.
 - Manejo de iluminación y texturas mediante shaders programables.



3.3 Librerías externas requeridas.

Librería	Función
GLFW	Creación de ventana y manejo de entrada de teclado y mouse.
GLEW	Carga de extensiones OpenGL.
GLM	Operaciones matemáticas y transformaciones.
stb_image	Carga de texturas.

Assimp	Importación de modelos 3D.
SOIL2	Manejo adicional de imágenes y texturas.

3.4 Requerimientos de hardware.

- GPU compatible con OpenGL 3.3+
- Memoria RAM de 8 GB recomendados.
- Procesador Intel i5 / Ryzen 3 o superior.
- Espacio suficiente para modelos, texturas y librerías.

3.5 Compatibilidad.

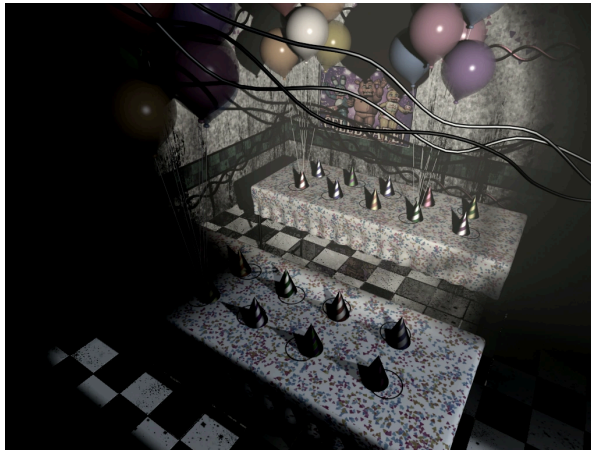
- Windows 10 / Windows 11.
- Arquitectura x86.
- Compatible con equipos y laptops de gama media.



3.6 Ambiente del proyecto.

El entorno virtual estará ambientado en una pizzería oscura inspirada en la estética de Five Nights at Freddy's, incorporando:

- Oficina de vigilancia.
- Pasillos y escenario principal.
- Mesas, sillas y decoración temática.
- Modelos 3D importados.
- Cámara en primera persona.
- Animaciones e interacción.



4. Arquitectura y Estructura del Proyecto (Visual Studio sin CMake)

El proyecto está desarrollado en C++ utilizando OpenGL moderno (Core Profile) y organizado bajo una arquitectura modular, permitiendo separar la lógica de renderizado, manejo de cámara, shaders, modelos y animaciones para facilitar el mantenimiento y escalabilidad del código.

La aplicación funciona mediante un ciclo principal de renderizado donde se procesan entradas del usuario, se actualizan animaciones y posteriormente se dibuja toda la escena 3D en tiempo real.

4.1 Arquitectura general del sistema.

El sistema está compuesto por los siguientes módulos principales:

Archivo / Módulo	Función
main.cpp	Contiene la lógica principal del programa, inicialización y renderizado.
Shader.h / Shader.cpp	Compilación, carga y manejo de shaders.
Camera.h / Camera.cpp	Control de cámara FPS con teclado y mouse.
Model.h / Model.cpp	Carga de modelos externos mediante Assimp.
Mesh.h / Mesh.cpp	Manejo interno de vértices, VAO, VBO y EBO.

Texture Loader	Carga y configuración de texturas.
Animation System	Actualización de movimientos y efectos básicos.

4.2 Flujo principal del programa.

El funcionamiento general del proyecto sigue la siguiente secuencia:

1. Inicialización de GLFW y GLEW.
2. Creación de ventana y contexto OpenGL.
3. Carga de shaders.
4. Configuración de cámara y proyección.
5. Carga de modelos y texturas.
6. Inicialización de iluminación.
7. Inicio del bucle principal:
 - Procesamiento de entrada (WASD + mouse).
 - Actualización de animaciones.
 - Configuración de luces.
 - Renderizado de la escena.
8. Liberación de recursos y cierre del programa.

4.3 Sistema de renderizado.

El renderizado utiliza:

- VAO/VBO/EBO para geometría.
- Matrices model, view y projection.
- Texturas cargadas mediante stb_image.
- Shaders GLSL para iluminación y materiales.
- Depth Testing para correcta profundidad.
- Renderizado en tiempo real a 60 FPS aproximados.

4.4 Cámara e interacción.

La navegación se realiza mediante:

- **Teclado:**
 - W, A, S, D → Movimiento.
- **Mouse:**
 - Rotación de cámara.
- **Teclas adicionales:**
 - Abrir puertas.
 - Activar animaciones de animatronicos.

La cámara funciona en primera persona para aumentar la inmersión del entorno de terror.

4.5 Organización visual del escenario

La escena estará dividida en zonas:

- Oficina principal
- Pasillo izquierdo.
- Pasillo derecho.
- Escenarios de animatrónicos.
- Área de mesas.

5. Metodología de Desarrollo.

5.1 Configuración inicial del proyecto.

El proyecto fue configurado en Visual Studio 2022 utilizando C++ y OpenGL 3.3 Core Profile, integrando librerías fundamentales como GLFW, GLEW, GLM, stb_image y Assimp para permitir la creación de ventanas, manejo de entrada del usuario, matemáticas gráficas, carga de texturas y modelos 3D externos. Esta etapa permitió establecer una base funcional para el desarrollo del entorno interactivo.

5.2 Construcción del escenario base.

Se desarrolló la estructura principal del escenario tomando inspiración en la ambientación de Five Nights at Freddy's, construyendo elementos como la oficina de seguridad, pasillos, escenario principal, mesas, sillas, paredes y puertas. Los objetos fueron posicionados mediante transformaciones de traslación, rotación y escala utilizando matrices de modelo para organizar correctamente el entorno tridimensional.

5.3 Aplicación de texturas y materiales.

Posteriormente se aplicaron texturas y materiales a los distintos objetos de la escena con el objetivo de generar una apariencia oscura y temática de terror. Se utilizaron superficies con apariencia metálica, madera envejecida y paredes deterioradas, además de configurarse propiedades de iluminación ambiental, difusa y especular para mejorar la respuesta visual de los materiales ante las fuentes de luz.

5.4 Implementación de iluminación.

La iluminación del proyecto fue diseñada para mejorar la ambientación del entorno utilizando múltiples luces puntuales distribuidas estratégicamente dentro del escenario, principalmente sobre las mesas y en la oficina. Se implementaron luces tipo Point Light, configurando propiedades de iluminación ambiente, difusa y especular para generar mayor profundidad y realismo en los objetos y modelos 3D. Además, se utilizó un Spotlight asociado a la cámara del usuario, simulando una linterna que sigue la dirección de visión durante el recorrido por el escenario. La iluminación fue controlada mediante shaders en OpenGL, utilizando transformaciones y vectores para calcular la interacción de la luz con las superficies, texturas y materiales del entorno en tiempo real.

5.5 Integración de modelos externos.

Se utilizaron modelos 3D externos importados mediante la librería Assimp, permitiendo incorporar animatrónicos, cámaras de vigilancia, objetos decorativos y mobiliario detallado. Los modelos fueron organizados dentro de carpetas específicas para mantener una estructura clara del proyecto y facilitar la administración de recursos durante el desarrollo.

5.6 Cámara e interacción del usuario.

El proyecto implementa una cámara en primera persona controlada mediante teclado y mouse, permitiendo al usuario recorrer libremente el entorno 3D. El movimiento se realiza utilizando las teclas WASD mientras que la orientación de la cámara se controla con el mouse, incorporando además teclas especiales para abrir puertas y activar los movimientos de los animatrónicos.

5.7 Animaciones y efectos básicos.

Se añadieron animaciones mediante transformaciones y el uso de deltaTime para mantener movimientos independientes de los FPS. Entre las animaciones implementadas se consideran movimiento de puertas, desplazamiento y rotación de animatrónicos y efectos visuales que ayudan a generar tensión dentro del entorno.

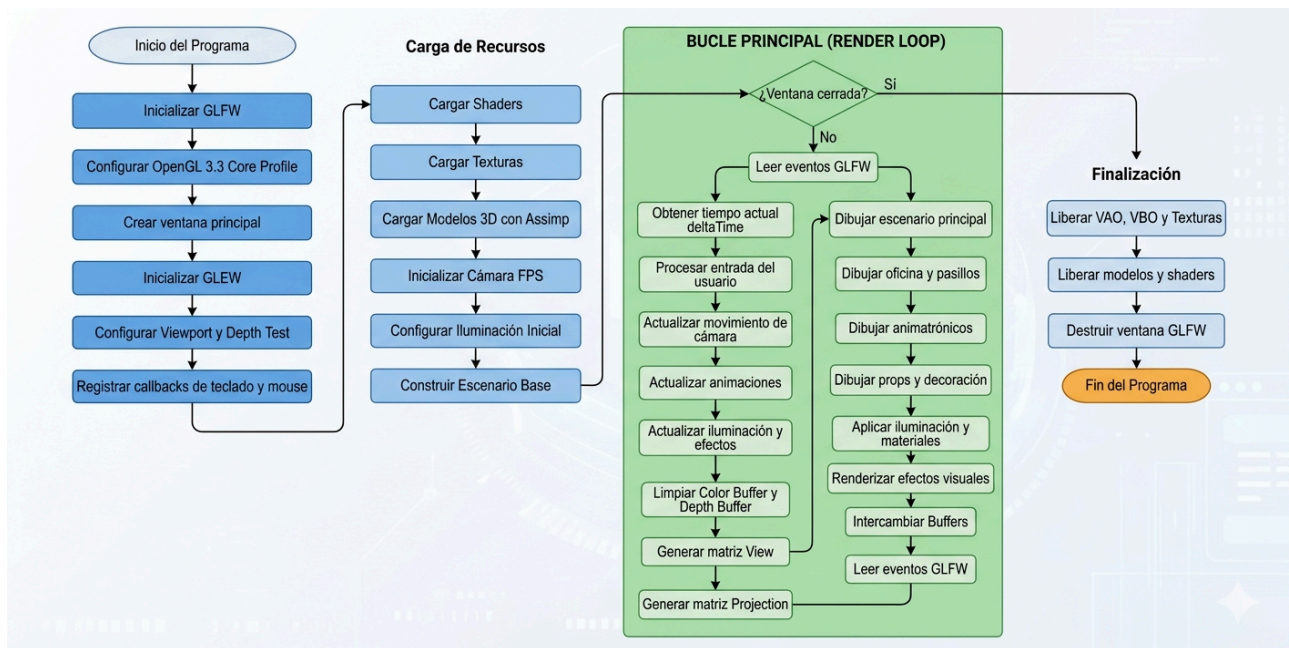
5.8 Pruebas y validación.

Durante el desarrollo se realizaron pruebas constantes para verificar la correcta carga de shaders, texturas y modelos, así como el funcionamiento de la cámara, iluminación y animaciones. También se validó el rendimiento general del proyecto con el fin de mantener estabilidad visual y fluidez durante la ejecución en tiempo real.

5.9 Documentación del proceso.

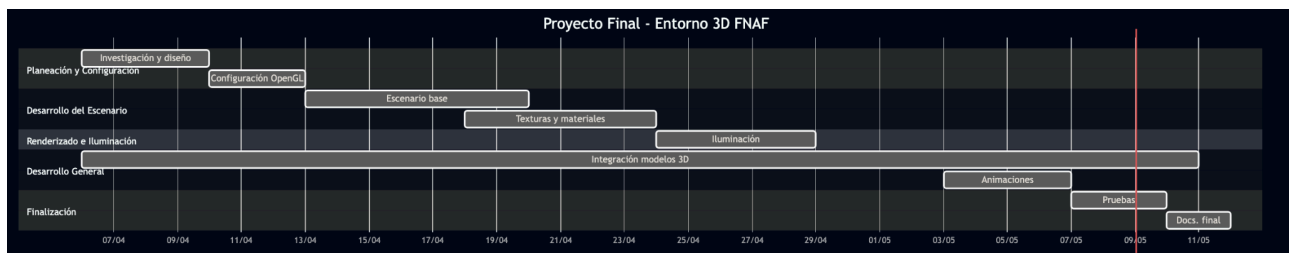
Finalmente, se documentó la estructura general del proyecto, la organización de archivos, el funcionamiento de los módulos principales y las decisiones técnicas tomadas durante el desarrollo. Esta documentación facilita la comprensión del código y permite futuras mejoras o ampliaciones del entorno virtual.

6. Diagrama de Flujo del Software.



7. Plan y Diagrama de Gantt.

El proyecto fue planeado en un periodo de 6 semanas comprendido entre el 6 de abril y el 11 de mayo. Durante este tiempo se distribuyeron las actividades de investigación, configuración del entorno gráfico, construcción del escenario, aplicación de texturas, implementación de iluminación, integración de modelos 3D, programación de interacción y animaciones, así como pruebas, optimización y documentación final. La organización temporal permitió desarrollar cada componente de manera progresiva y mantener control sobre el avance general del proyecto.



Actividad	Responsable	Inicio	Fin
Configuración de Visual Studio, OpenGL, GLFW, GLEW y Assimp	Nikole	S1, d1	S1, d4
Organización de carpetas y estructura del proyecto	Nikole	S1, d2	S1, d6
Construcción del escenario base (oficina, pasillos y escenario)	Berenice	S2, d1	S2, d7

Modelado y ajuste de props en Blender	Berenice	S2, d3	S3, d4
Integración de modelos 3D en OpenGL (Assimp)	Equipo	S3, d1	S6, d7
Aplicación de texturas y materiales	Equipo	S3, d4	S4, d5
Implementación de iluminación y shaders	Berenice	S4, d1	S4, d7
Programación de animaciones y efectos visuales	Nikole	S5, d1	S5, d6
Pruebas, corrección de errores y optimización	Equipo	S6, d1	S6, d4
Documentación final y preparación de exposición	Equipo	S6, d3	S6, d7

8. Documentación del Código.

8.1 Librerías y clases principales.

En esta sección se integran todas las librerías necesarias para el funcionamiento del proyecto en OpenGL. GLFW permite crear la ventana y detectar eventos de teclado y mouse, mientras que GLEW habilita el uso de funciones modernas de OpenGL. GLM se utiliza para realizar operaciones matemáticas relacionadas con matrices, vectores y transformaciones tridimensionales. También se incorporan librerías para cargar texturas y modelos externos, como stb_image, SOIL2 y Assimp. Finalmente, las clases Shader, Camera y Model ayudan a organizar el código y facilitar el manejo de shaders, cámara y modelos 3D dentro de la escena.

```

1  #include <iostream>
2  #include <cmath>
3
4  // GLEW
5  #include <GL/glew.h>
6
7  // GLFW
8  #include <GLFW/glfw3.h>
9
10 // Other Libs
11 #include "stb_image.h"
12
13 // GLM Mathematics
14 #include <glm/glm.hpp>
15 #include <glm/gtc/matrix_transform.hpp>
16 #include <glm/gtc/type_ptr.hpp>
17
18 //Load Models
19 #include "SOIL2/SOIL2.h"
20
21
22 // Other includes
23 #include "Shader.h"
24 #include "Camera.h"
25 #include "Model.h"

```

8.2 Configuración general de ventana y cámara.

Esta parte del código define el tamaño de la ventana y configura la cámara principal del proyecto. La cámara se inicializa en una posición específica frente al escenario para permitir la navegación dentro del entorno 3D. También se utilizan variables auxiliares para controlar el movimiento del mouse y detectar la primera interacción del usuario. El arreglo de teclas permite identificar qué teclas están presionadas durante la ejecución del programa. Todo esto es fundamental para crear una experiencia interactiva en primera persona similar a un videojuego.

```
26 // Function prototypes
27 void KeyCallback(GLFWwindow* window, int key, int scancode, int action, int mode);
28 void MouseCallback(GLFWwindow* window, double xPos, double yPos);
29 void DoMovement();
30 void Animation();
31
32 // Window dimensions
33 const GLuint WIDTH = 800, HEIGHT = 600;
34 int SCREEN_WIDTH, SCREEN_HEIGHT;
35
36 // Camera
37 Camera camera(glm::vec3(0.0f, 0.0f, 3.0f));
38 GLfloat lastX = WIDTH / 2.0;
39 GLfloat lastY = HEIGHT / 2.0;
40 bool keys[1024];
41 bool firstMouse = true;
42 // Light attributes
43 glm::vec3 lightPos(0.0f, 0.0f, 0.0f);
44 bool active;
45
46 // Positions of the point lights
47 glm::vec3 pointLightPositions[] = {
48     glm::vec3(0.0f, 2.0f, 0.0f),
49     glm::vec3(0.0f, 0.0f, 0.0f),
50     glm::vec3(0.0f, 0.0f, 0.0f),
51     glm::vec3(0.0f, 0.0f, 0.0f)
52 }
```

8.3 Variables de iluminación.

Las variables de iluminación permiten controlar la posición, color y comportamiento de las luces utilizadas dentro del escenario. Se implementa una luz puntual que puede activarse o desactivarse mediante el teclado para generar diferentes efectos visuales. Además, el color de la luz cambia dinámicamente utilizando funciones matemáticas basadas en el tiempo. Estas variables ayudan a crear una atmósfera oscura y de suspenso inspirada en Five Nights at Freddy's. La iluminación es uno de los elementos más importantes para mejorar el ambiente visual del proyecto.

8.4 Variables de animación.

En esta sección se almacenan las variables utilizadas para controlar las animaciones del escenario. Cada objeto animado cuenta con una variable numérica que define su movimiento o rotación y una variable booleana que controla si la animación está activa. Entre las animaciones implementadas se encuentran el movimiento del sol y la nube, el rebote de la pelota, la rotación de las aspas del ventilador y la apertura de puertas. Estas animaciones permiten que la escena tenga movimiento constante y no se perciba completamente estática. Gracias a esto, el entorno resulta más dinámico e inmersivo.

```

glm::vec3 Light1 = glm::vec3(0);

//Anim Basicas
//Sol y Nube
float movSol = 0;
bool AnimSol = true;
//Pelota
float movPelota = 0;
bool AnimPelota = false;
//Aspas
float rotAspas = 0.0f;
bool AnimAspas = false;
//PuertaDercha
float movPuertaD = 0;
bool AnimPuertaD = false;
//PuertaIzquierda
float movPuertaI = 0;
bool AnimPuertaI = false;
////Anim Complejas
/////Personaje Puppet
//float movPuppet = 0.0f;
//float rotBrazoDerP = 0.0f;
//float rotBrazoIzqP = 0.0f;
//float rotPiernaDerP = 0.0f;
//float rotPiernaIzqP = 0.0f;
//bool AnimPuppet = false;
//bool adelante = true;
/////Personaje Chica
///bool AnimBall = false;

// Deltatime
GLfloat deltaTime = 0.0f; // Time between curr
GLfloat lastFrame = 0.0f; // Time of last frame

```

8.5 Inicialización de GLFW, GLEW y OpenGL.

Esta sección configura el entorno gráfico necesario para ejecutar el proyecto utilizando OpenGL moderno. Primero se inicializa GLFW y se crea la ventana principal del programa con soporte para OpenGL 3.3 Core Profile. Posteriormente se inicializa GLEW para acceder a funciones avanzadas de renderizado y shaders. También se configura el viewport y se activa el Depth Test, que permite dibujar correctamente la profundidad de los objetos dentro de la escena. Esta etapa es fundamental porque prepara todo el contexto gráfico antes de comenzar el renderizado.

```

// Init GLFW
glfwInit();
// Set all the required options for GLFW
//glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
glfwWindowHint(GLFW_RESIZABLE, GL_FALSE);

// Create a GLFWwindow object that we can use for GLFW's functions
GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "Animacion basica", nullptr, nullptr);

if (nullptr == window)
{
    std::cout << "Failed to create GLFW window" << std::endl;
    glfwTerminate();
    return EXIT_FAILURE;
}

glfwMakeContextCurrent(window);

glfwGetFramebufferSize(window, &SCREEN_WIDTH, &SCREEN_HEIGHT);

// Set the required callback functions
glfwSetKeyCallback(window, KeyCallback);
glfwSetCursorPosCallback(window, MouseCallback);

// GLFW Options
//glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);

// Set this to true so GLEW knows to use a modern approach to retrieving function pointers and extensions
glewExperimental = GL_TRUE;
// Initialize GLEW to setup the OpenGL Function pointers
if (GLEW_OK != glewInit())
{

```

8.6 Carga de shaders.

Los shaders son programas que se ejecutan directamente en la GPU y controlan el comportamiento visual de los objetos. En este proyecto se utilizan dos shaders principales: uno para la iluminación general del escenario y otro para dibujar el cubo indicador de luz. Estos shaders permiten calcular materiales, iluminación, reflejos y colores dentro de la escena 3D. Gracias a ellos es posible implementar luces direccionales, luces puntuales y spotlight para mejorar la atmósfera del entorno. La carga correcta de shaders es esencial para lograr un renderizado moderno en OpenGL.

```
Shader lightingShader("Shader/lighting.vs", "Shader/lighting.frag");
Shader lampShader("Shader/lamp.vs", "Shader/lamp.frag");
```

8.7 Carga de modelos 3D del escenario.

Esta parte del código se encarga de importar todos los modelos 3D utilizados en el proyecto. Los modelos fueron creados previamente en software de modelado y exportados en formato .obj para ser cargados mediante la clase Model. Entre los objetos utilizados se encuentran el mapa principal, puertas, ventilador, globos, sillas, vasos y animatrónicos. Cada modelo es almacenado individualmente para poder dibujarse posteriormente dentro del escenario. Esto permite construir una escena compleja y organizada utilizando múltiples objetos independientes.

```
//models
Model ModelosIntegrados((char*)"Models/mdlMapa.obj");
Model Sol((char*)"Models/mdlSol.obj");
Model Nube((char*)"Models/mdlNube.obj");
Model Bocina((char*)"Models/mdlBocina.obj");
Model Pelota((char*)"Models/mdlPelota.obj");
Model Globos((char*)"Models/mdlGlobos.obj");
Model Sillas((char*)"Models/mdlSillas.obj");
Model Vaso((char*)"Models/mdlVaso.obj");
Model PuertaDerecha((char*)"Models/mdlPuertaDerecha.obj");
Model PuertaIzquierda((char*)"Models/mdlPuertaIzquierda.obj");
Model Ventilador((char*)"Models/mdlVentilador.obj");
Model Aspas((char*)"Models/mdlAspas.obj");
Model Animatronicos((char*)"Models/mdlAnimatronicos.obj");
```



```

//Carga de modelo
//Mapa
model = glm::mat4(1);
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
glUniformli(glGetUniformLocation(lightningShader.Program, "transparency"), 0);
ModelosIntegrados.Draw(lightningShader);
//Sol
model = glm::mat4(1);
model = glm::translate(model, glm::vec3(0.0f, movSol, 0.0f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glUniformli(glGetUniformLocation(lightningShader.Program, "transparency"), 0);
Sol.Draw(lightningShader);
//Nube
model = glm::mat4(1);
model = glm::translate(model, glm::vec3(0.0f, -movSol, 0.0f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glUniformli(glGetUniformLocation(lightningShader.Program, "transparency"), 0);
Nube.Draw(lightningShader);
//Bocina
model = glm::mat4(1);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glUniformli(glGetUniformLocation(lightningShader.Program, "transparency"), 0);
Bocina.Draw(lightningShader);
///Pelota
model = glm::mat4(1);
model = glm::translate(model, glm::vec3(0.0f, movPelota, 0.0f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glUniformli(glGetUniformLocation(lightningShader.Program, "transparency"), 0);
Pelota.Draw(lightningShader);
//Globos
model = glm::mat4(1);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glUniformli(glGetUniformLocation(lightningShader.Program, "transparency"), 0);
Globos.Draw(lightningShader);
//Sillas

```

Carga de modelos Open GL

Este código define y dibuja varios cubos en un entorno 3D utilizando OpenGL y la librería GLM. Primero, el arreglo `vertices[]` almacena la información de los vértices de un cubo, incluyendo sus posiciones en el espacio (x, y, z), las normales de cada cara para calcular la iluminación y las coordenadas de textura para aplicar imágenes sobre las superficies. Después, el arreglo `cubePositions[]` contiene las posiciones donde se colocará cada cubo dentro de la escena, mientras que `cubeScales[]` define el tamaño y proporción de cada uno, permitiendo representar distintos objetos como pizzas, cajas, regalos, hojas, pósters y pantallas. En el ciclo `for`, el programa recorre los 17 cubos y para cada uno crea una matriz de transformación llamada `model`, la cual aplica primero una traslación para posicionar el cubo y luego una escala para modificar su tamaño. Posteriormente, se envía esta matriz al shader con `glUniformMatrix4fv`, se activa y enlaza la textura correspondiente usando `glBindTexture`, y finalmente se dibuja el cubo mediante `glDrawArrays(GL_TRIANGLES, 0, 36)`, donde los 36 vértices representan los 12 triángulos que forman las 6 caras del cubo.

```

// Dibujar los 7 cubos
for (GLuint i = 0; i < 17; i++)
{
    model = glm::mat4(1);

    // Posición
    model = glm::translate(model, cubePositions[i]);

    // Escala
    model = glm::scale(model, cubeScales[i]);

    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

    // Enviar color al shader
    /*GLuint colorLoc = glGetUniformLocation(lampShader.Program, "objectColor");
    glUniform3fv(colorLoc, 1, glm::value_ptr(cubeColors[i]));*/

    glBindVertexArray(VAO);*/

    //ACTIVAR TEXTURA

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, cubeTextures[i]);
    glUniform1i(glGetUniformLocation(lampShader.Program, "texture1"), 0);

    glBindVertexArray(VAO);
    glDrawArrays(GL_TRIANGLES, 0, 36);
}

```

```

float vertices[] = {
    // posiciones           // normales           // textura
    -0.5f,-0.5f,-0.5f,      0.0f, 0.0f,-1.0f,      0.0f, 0.0f,
    0.5f,-0.5f,-0.5f,      0.0f, 0.0f,-1.0f,      1.0f, 0.0f,
    0.5f, 0.5f,-0.5f,      0.0f, 0.0f,-1.0f,      1.0f, 1.0f,
    -0.5f, 0.5f,-0.5f,      0.0f, 0.0f,-1.0f,      1.0f, 1.0f,
    -0.5f, 0.5f,-0.5f,      0.0f, 0.0f,-1.0f,      0.0f, 1.0f,
    -0.5f,-0.5f,-0.5f,      0.0f, 0.0f,-1.0f,      0.0f, 0.0f,

    -0.5f,-0.5f, 0.5f,      0.0f, 0.0f, 1.0f,      0.0f, 0.0f,
    0.5f,-0.5f, 0.5f,      0.0f, 0.0f, 1.0f,      1.0f, 0.0f,
    0.5f, 0.5f, 0.5f,      0.0f, 0.0f, 1.0f,      1.0f, 1.0f,
    0.5f, 0.5f, 0.5f,      0.0f, 0.0f, 1.0f,      1.0f, 1.0f,
    -0.5f, 0.5f, 0.5f,      0.0f, 0.0f, 1.0f,      0.0f, 1.0f,
    -0.5f,-0.5f, 0.5f,      0.0f, 0.0f, 1.0f,      0.0f, 0.0f,

    -0.5f, 0.5f, 0.5f,      -1.0f, 0.0f, 0.0f,      1.0f, 0.0f,
    -0.5f, 0.5f,-0.5f,      -1.0f, 0.0f, 0.0f,      1.0f, 1.0f,
    -0.5f,-0.5f,-0.5f,      -1.0f, 0.0f, 0.0f,      0.0f, 1.0f,
    -0.5f,-0.5f,-0.5f,      -1.0f, 0.0f, 0.0f,      0.0f, 1.0f,
    -0.5f,-0.5f, 0.5f,      -1.0f, 0.0f, 0.0f,      0.0f, 0.0f,
    -0.5f, 0.5f, 0.5f,      -1.0f, 0.0f, 0.0f,      1.0f, 0.0f,

    0.5f, 0.5f, 0.5f,      1.0f, 0.0f, 0.0f,      1.0f, 0.0f,
    0.5f, 0.5f,-0.5f,      1.0f, 0.0f, 0.0f,      1.0f, 1.0f,
    0.5f,-0.5f,-0.5f,      1.0f, 0.0f, 0.0f,      0.0f, 1.0f,
    0.5f,-0.5f,-0.5f,      1.0f, 0.0f, 0.0f,      0.0f, 1.0f,
}

```

8.8 Bucle principal de renderizado.

El bucle principal es la parte más importante del programa, ya que controla todo el funcionamiento en tiempo real del entorno 3D. En cada iteración se calcula el tiempo transcurrido entre frames, se procesan eventos de teclado y mouse y se actualizan las animaciones. Después se limpia la pantalla y se renderizan nuevamente todos los objetos del escenario. Este proceso se repite constantemente mientras la ventana permanezca abierta. Gracias a este ciclo continuo, el usuario puede interactuar con la escena de forma fluida y dinámica.

```
// Game loop
while (!glfwWindowShouldClose(window))
{
    // Calculate deltatime of current frame
    GLfloat currentFrame = glfwGetTime();
    deltaTime = currentFrame - lastFrame;
    lastFrame = currentFrame;

    // Check if any events have been activated (key pressed, mouse moved etc.) and call corresponding response functions
    glfwPollEvents();
    DoMovement();
    Animation();

    // Clear the colorbuffer
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // OpenGL options
    glEnable(GL_DEPTH_TEST);

    // Use cooresponding shader when setting uniforms/drawing objects
    lightingShader.Use();

    glUniform1i(glGetUniformLocation(lightingShader.Program, "diffuse"), 0);
    //glUniform1i(glGetUniformLocation(lightingShader.Program, "specular"),1);

    GLint viewPosLoc = glGetUniformLocation(lightingShader.Program, "viewPos");
    glUniform3f(viewPosLoc, camera.GetPosition().x, camera.GetPosition().y, camera.GetPosition().z);

    // Directional light
    glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.direction"), -0.2f, -1.0f, -0.3f);
    glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.ambient"), 0.6f, 0.6f, 0.6f);
    glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.diffuse"), 0.6f, 0.6f, 0.6f);
    glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.specular"), 0.3f, 0.3f, 0.3f);
}
```

8.9 Configuración de luces: direccional, puntual y spotlight.

El proyecto implementa distintos tipos de iluminación para mejorar la ambientación visual del escenario. La luz direccional funciona como iluminación general para todo el mapa, mientras que la luz puntual genera efectos localizados y dinámicos. Además, se utiliza un spotlight conectado a la cámara que funciona como linterna y apunta hacia donde mira el usuario. Cada luz cuenta con propiedades ambientales, difusas y especulares para controlar su comportamiento sobre los materiales. Esta combinación de luces ayuda a crear una atmósfera oscura y más cercana a la temática de terror del proyecto.

```

GLint viewPosLoc = glGetUniformLocation(LightingShader.Program, "viewPos");
glUniform3f(viewPosLoc, camera.GetPosition().x, camera.GetPosition().y, camera.GetPosition().z);

// Directional light
glUniform3f(glGetUniformLocation(LightingShader.Program, "dirLight.direction"), -0.2f, -1.0f, -0.3f);
glUniform3f(glGetUniformLocation(LightingShader.Program, "dirLight.ambient"), 0.6f, 0.6f, 0.6f);
glUniform3f(glGetUniformLocation(LightingShader.Program, "dirLight.diffuse"), 0.6f, 0.6f, 0.6f);
glUniform3f(glGetUniformLocation(LightingShader.Program, "dirLight.specular"), 0.3f, 0.3f, 0.3f);

// Point light 1
glm::vec3 lightColor;
lightColor.x = abs(sin(glfwGetTime() * Light1.x));
lightColor.y = abs(sin(glfwGetTime() * Light1.y));
lightColor.z = sin(glfwGetTime() * Light1.z);

glUniform3f(glGetUniformLocation(LightingShader.Program, "pointLights[0].position"), pointLightPositions[0].x, pointLightPositions[0].y, pointLightPositions[0].z);
glUniform3f(glGetUniformLocation(LightingShader.Program, "pointLights[0].ambient"), lightColor.x, lightColor.y, lightColor.z);
glUniform3f(glGetUniformLocation(LightingShader.Program, "pointLights[0].diffuse"), lightColor.x, lightColor.y, lightColor.z);
glUniform3f(glGetUniformLocation(LightingShader.Program, "pointLights[0].specular"), 1.0f, 0.2f, 0.2f);
glUniform1f(glGetUniformLocation(LightingShader.Program, "pointLights[0].constant"), 1.0f);
glUniform1f(glGetUniformLocation(LightingShader.Program, "pointLights[0].linear"), 0.045f);
glUniform1f(glGetUniformLocation(LightingShader.Program, "pointLights[0].quadratic"), 0.075f);

// SpotLight
glUniform3f(glGetUniformLocation(LightingShader.Program, "spotLight.position"), camera.GetPosition().x, camera.GetPosition().y, camera.GetPosition().z);
glUniform3f(glGetUniformLocation(LightingShader.Program, "spotLight.direction"), camera.GetFront().x, camera.GetFront().y, camera.GetFront().z);
glUniform3f(glGetUniformLocation(LightingShader.Program, "spotLight.ambient"), 0.2f, 0.2f, 0.8f);
glUniform3f(glGetUniformLocation(LightingShader.Program, "spotLight.diffuse"), 0.2f, 0.2f, 0.8f);
glUniform3f(glGetUniformLocation(LightingShader.Program, "spotLight.specular"), 0.0f, 0.0f, 0.0f);
glUniform1f(glGetUniformLocation(LightingShader.Program, "spotLight.constant"), 1.0f);
glUniform1f(glGetUniformLocation(LightingShader.Program, "spotLight.linear"), 0.3f);
glUniform1f(glGetUniformLocation(LightingShader.Program, "spotLight.quadratic"), 0.7f);
glUniform1f(glGetUniformLocation(LightingShader.Program, "spotLight.cutOff"), glm::cos(glm::radians(12.0f)));
glUniform1f(glGetUniformLocation(LightingShader.Program, "spotLight.outerCutoff"), glm::cos(glm::radians(30.0f)));

```

8.10 Sistema de interacción y animaciones.

El sistema de interacción permite que el usuario controle distintos elementos del entorno mediante teclado y mouse. La cámara puede desplazarse libremente utilizando WASD y el mouse permite cambiar la dirección de visión dentro de la escena. También existen teclas específicas para abrir puertas, activar el ventilador o encender y apagar luces dinámicas. Por otro lado, el sistema de animaciones actualiza continuamente el movimiento de objetos como la pelota, las puertas y las aspas del ventilador. Esto hace que el escenario se sienta más interactivo y visualmente activo.

```

570 // Moves/alters the camera positions based on user input
571 void DoMovement()
572 {
573     // Camera controls
574     if (keys[GLFW_KEY_W] || keys[GLFW_KEY_UP])
575     {
576         camera.ProcessKeyboard(FORWARD, deltaTime);
577     }
578
579     if (keys[GLFW_KEY_S] || keys[GLFW_KEY_DOWN])
580     {
581         camera.ProcessKeyboard(BACKWARD, deltaTime);
582     }
583
584     if (keys[GLFW_KEY_A] || keys[GLFW_KEY_LEFT])
585     {
586         camera.ProcessKeyboard(LEFT, deltaTime);
587     }
588
589     if (keys[GLFW_KEY_D] || keys[GLFW_KEY_RIGHT])
590     {
591         camera.ProcessKeyboard(RIGHT, deltaTime);
592     }
593
594     if (keys[GLFW_KEY_T])
595     {
596         pointLightPositions[0].x += 0.01f;
597     }
598     if (keys[GLFW_KEY_G])
599     {
600         pointLightPositions[0].x -= 0.01f;
601     }
602 }

```

9. Diseño del Escenario 3D (Geometría, Texturizado, Iluminación, Animación)



El escenario 3D del proyecto fue diseñado como una pizzería oscura inspirada en Five Nights at Freddy's, integrando elementos visuales característicos como pasillos, puertas laterales, mobiliario, decoración, ventilador, globos, bocinas y animatrónicos. La escena se construye principalmente a partir de modelos externos cargados en formato .obj, los cuales representan la estructura completa del mapa y los objetos principales del entorno. Esta organización permite separar cada elemento importante para poder modificarlo, transformarlo o animarlo de manera independiente dentro del programa.

En cuanto a la **geometría**, el proyecto utiliza modelos 3D integrados mediante la clase Model, incluyendo el mapa principal, puertas, sillas, vaso, ventilador, aspas, pelota, sol, nube, globos, bocina y animatrónicos. Cada objeto se coloca dentro de la escena mediante matrices de transformación, principalmente utilizando glm::mat4, glm::translate, glm::scale y glm::rotate. Esto permite posicionar, mover, escalar o rotar elementos específicos sin alterar el modelo original, facilitando la construcción de una escena modular y organizada.

El **texturizado** se maneja a través de los modelos importados y sus materiales asociados. Cada archivo .obj puede incluir texturas y coordenadas UV que permiten representar superficies como paredes, pisos, objetos decorativos, mobiliario y animatrónicos. El shader principal utiliza propiedades de material como textura difusa, textura especular y brillo para mejorar la apariencia visual de los modelos. Esto permite diferenciar materiales dentro del escenario y dar mayor realismo a la pizzería.

La **iluminación** es uno de los elementos más importantes del proyecto, ya que refuerza la atmósfera de suspenso. Se implementa una luz direccional para iluminar de forma

general el escenario, una luz puntual dinámica que puede activarse con la tecla espacio y un spotlight conectado a la cámara, simulando una linterna. Estas luces trabajan con componentes ambientales, difusas y especulares, además de parámetros de atenuación para controlar la intensidad según la distancia. Gracias a esto, la escena mantiene una estética oscura, pero funcional para la exploración.

Finalmente, la **animación** se implementa mediante variables de movimiento y rotación actualizadas en la función Animation(). El sol y la nube se desplazan de forma opuesta, la pelota realiza un movimiento vertical tipo rebote, las aspas del ventilador giran cuando se activan y las puertas laterales se abren o cierran gradualmente mediante teclas. Estas animaciones se basan en incrementos matemáticos simples y no requieren un motor de físicas, pero aportan dinamismo e interacción al escenario.

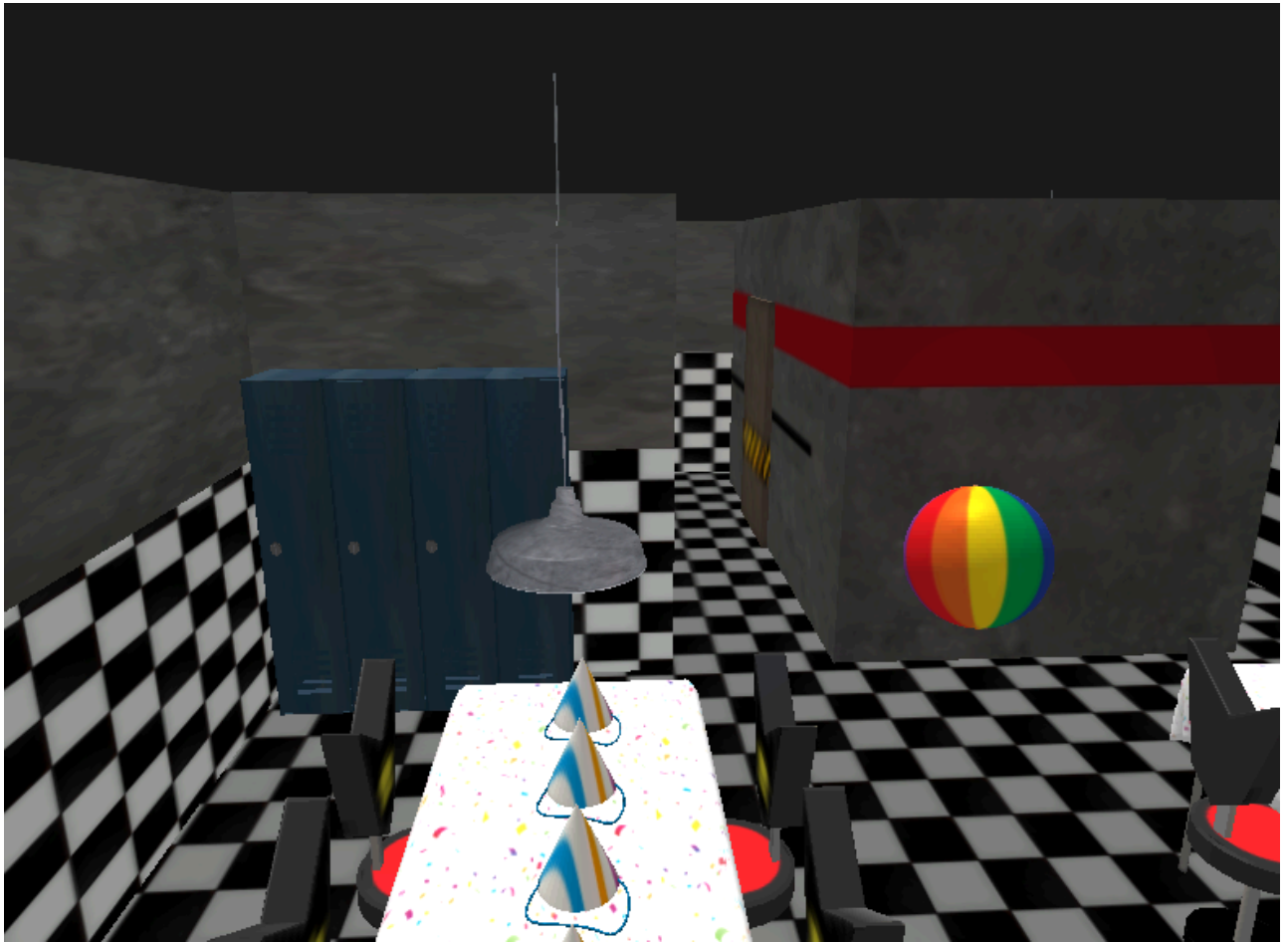
10. Pruebas y Validación.

Durante el desarrollo del proyecto se realizaron pruebas para verificar la correcta carga de shaders, modelos 3D y texturas dentro de la escena. También se validó el funcionamiento de la cámara en primera persona, la iluminación y las animaciones implementadas. Se revisó que las teclas de interacción respondieran correctamente y que el renderizado funcionara de manera estable en tiempo real. Finalmente, se realizaron pruebas de rendimiento para mantener una ejecución fluida en equipos de nivel medio.











11. Gestión de Riesgos y Mantenimiento.

Uno de los principales riesgos del proyecto es la incorrecta configuración de rutas para modelos, texturas o shaders, lo que puede provocar errores de carga durante la ejecución. También pueden existir problemas relacionados con librerías externas o drivers gráficos incompatibles con OpenGL 3.3. Para reducir estos riesgos, el proyecto fue organizado en carpetas específicas y el código fue comentado y dividido por secciones. Esto facilita el mantenimiento, la corrección de errores y futuras mejoras dentro del entorno 3D.

12. Conclusiones y Trabajo Futuro.

El desarrollo del proyecto permitió integrar distintos conceptos fundamentales de computación gráfica moderna utilizando OpenGL, como iluminación, texturizado, modelos 3D, transformaciones, cámara en primera persona y animaciones básicas. A través de este entorno inspirado en Five Nights at Freddy's fue posible comprender mejor el funcionamiento del pipeline gráfico, el uso de shaders y la interacción entre CPU y GPU durante el renderizado en tiempo real. Además, el proyecto ayudó a reforzar habilidades

relacionadas con organización de código, manejo de librerías externas y construcción de escenas tridimensionales interactivas.

La implementación de iluminación direccional, luz puntual y spotlight permitió mejorar considerablemente la atmósfera visual del escenario, mientras que las animaciones y controles de interacción ayudaron a que la escena se sintiera más dinámica e inmersiva. También se logró integrar correctamente modelos externos mediante Assimp y organizar los recursos del proyecto de forma modular y estructurada. En conjunto, el proyecto representa una experiencia completa de desarrollo gráfico en OpenGL aplicada a un entorno temático de terror.

Como trabajo futuro, el proyecto podría ampliarse incorporando sombras dinámicas, mejores materiales y técnicas de renderizado más avanzadas como PBR o postprocesado. También podrían añadirse colisiones, inteligencia artificial básica para los animatrónicos, sonido ambiental y un sistema de cámaras de vigilancia más interactivo. Finalmente, sería posible optimizar aún más el rendimiento del escenario y expandir el mapa con nuevas habitaciones, objetos y mecánicas inspiradas en videojuegos de terror.